

SISTEMA BUMAND

Plataforma de Gestión de Becarios Universitarios

Manual de Arquitectura Técnica, Documentación Swagger y Especificación de la API REST

*Tratado integral de ingeniería de software para el backend institucional:
arquitectura modular en NestJS, modelo relacional de 15 tablas, persistencia
TypeORM, seguridad RBAC, algoritmos geoespaciales, liquidación en centavos y
documentación OpenAPI 3.0 interactiva.*

Fundación Diaconía Fondo Rotativo de Inversión y Fomento (FRIF-IFD)

Departamento de Desarrollo Tecnológico e Innovación

Versión 1.0 — La Paz, Bolivia — Octubre 2026

Índice

1. Fundamentos Arquitectónicos y Filosofía del Sistema	2
2. Estructura de Directorios del Código Fuente	2
3. Modelo de Datos Relacional de 15 Tablas	2
4. Reglas de Negocio Críticas y Algoritmos	2
4.1. Cálculo de Reembolso del 80 % en Centavos Enteros	2
4.2. Restricción Temporal: Regla Institucional del Día 24	3
4.3. Delimitación de Geocercas con Fórmula de Haversine	3
4.4. Matriz 360° Ponderada y Formulario Pastoral F-03	3
5. Documentación Interactiva OpenAPI y Swagger (/api/docs)	3
6. Aseguramiento de Calidad y Verificación	4

1. Fundamentos Arquitectónicos y Filosofía del Sistema

El subsistema backend de BUMAND ha sido concebido para modernizar de forma definitiva la administración del Programa de Becarios Universitarios de la **Fundación Diaconía FRIF-IFD**. La transición desde procesos físicos basados en planillas impresas hacia un monolito modular altamente escalable garantiza integridad transaccional, precisión monetaria al centavo y auditoría inmutable.

La arquitectura se fundamenta en los siguientes pilares de diseño:

- **Monolito Modular NestJS 11:** Inversión de control estricta con inyección de dependencias por constructor tipado en TypeScript 5.9.
- **Español Absoluto en el Dominio:** Nomenclatura corporativa en español para clases, DTOs, entidades, decoradores, columnas y rutas REST.
- **Seguridad Perimetral y RBAC:** Validación global mediante `ValidationPipe`, autenticación JWT sin estado y guardias de control de acceso basado en roles (`administrador`, `supervisor`, `becario`).
- **Persistencia y Migraciones SQL:** Motor PostgreSQL 15+ administrado por TypeORM 1.1 con esquema relacional canónico de 15 tablas e índices compuestos.

2. Estructura de Directorios del Código Fuente

El directorio `bumand-backend/src/` organiza la totalidad de las responsabilidades funcionales y de soporte:

- `main.ts`: Punto de entrada, tubería de validación global y servidor de documentación interactiva Swagger.
- `app.modulo.ts`: Enlace maestro de configuración, conexión a base de datos y módulos de dominio.
- `utilidades/`: Algoritmos canónicos de geolocalización (`geo.utilidad.ts`) y cifrado (`hash.utilidad.ts`).
- `datos/`: DDL canónico de 15 tablas (`001_esquema_completo_15_tablas.sql`) y servicio semilla idempotente (`semilla.servicio.ts`).
- `modulos/`: Módulos de dominio funcional (`autenticacion`, `usuarios`, `becarios`, `iglesias`, `lugares-practica`, `asistencia`, `pasajes`, `evaluaciones`, `practicas`, `notificaciones`, `dashboard`, `pdf`).

3. Modelo de Datos Relacional de 15 Tablas

El esquema de persistencia se estructura en 15 tablas con restricciones de clave foránea e índices compuestos para optimizar lecturas analíticas:

4. Reglas de Negocio Críticas y Algoritmos

4.1. Cálculo de Reembolso del 80 % en Centavos Enteros

Para prevenir desajustes monetarios generados por la aritmética de coma flotante de doble precisión (estándar IEEE 754), el cálculo se realiza en centavos enteros:

$$M_{\text{centavos}} = \text{round}(T \cdot 100), \quad R_{\text{centavos}} = \left\lfloor \frac{M_{\text{centavos}} \cdot 80}{100} \right\rfloor, \quad R = \frac{R_{\text{centavos}}}{100} \quad (1)$$

#	Tabla Relacional	Propósito y Atributos Principales
1	usuarios	Cuentas, correo institucional único, hash bcrypt y rol.
2	iglesias	Directorio de congregaciones cristianas vinculadas.
3	lugares_practica	Sedes operativas, coordenadas WGS84 y radio de tolerancia.
4	becarios	Expediente universitario, carrera, tutor e iglesia asignada.
5	registros_asistencia	Marcaciones de ingreso/salida, GPS, horas computadas.
6	solicitudes_pasajes	Liquidación mensual, total declarado y reembolso del 80 %.
7	recorridos	Tramos diarios de ida y vuelta con tarifa y coordenadas.
8	modulos_evaluacion	Catálogo de los 5 módulos de formación y evaluación 360°.
9	periodos_evaluacion	Semestres académicos vigentes e históricos.
10	evaluadores	Catálogo de actores habilitados para calificar.
11	evaluaciones	Registro de calificaciones numéricas y observaciones.
12	evaluaciones_pastor	Extensión semi-estructurada (JSONB) del formulario F-03.
13	aprobaciones_practicas	Convalidación de horas y firma digital en Base64.
14	notificaciones	Bandeja de avisos reactivos y notificaciones push.
15	reportes_generados	Registro y auditoría de documentos PDF emitidos.

Tabla 1: Matriz del esquema relacional institucional de 15 tablas.

4.2. Restricción Temporal: Regla Institucional del Día 24

Las solicitudes mensuales de viáticos no pueden ser presentadas a revisión antes del día 24 del mes de reporte. El servicio valida la fecha del sistema rechazando intentos prematuros mediante una excepción `BadRequestException`.

4.3. Delimitación de Geocercas con Fórmula de Haversine

La validación geoespacial calcula la distancia ortodrómica sobre la esfera terrestre entre la coordenada reportada por el sensor móvil y el centroide de la sede asignada:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cos(\phi_2) \sin^2\left(\frac{\Delta\lambda}{2}\right), \quad d = 2R \arcsin(\sqrt{a}) \quad (2)$$

con $R = 6,371,000$ m. Si $d \leq \text{radio_tolerancia_m}$, la marcación es admitida.

4.4. Matriz 360° Ponderada y Formulario Pastoral F-03

La calificación global semestral se pondera sobre 100 puntos y se proyecta a la escala vigesimal (0 a 20 puntos):

$$N_{\text{total}} = 0,25 \cdot N_{\text{F03}} + 0,20 \cdot N_{\text{Auto}} + 0,25 \cdot N_{\text{Mentor}} + 0,15 \cdot N_{\text{Lideres}} + 0,15 \cdot N_{\text{Socioecon}} \quad (3)$$

5. Documentación Interactiva OpenAPI y Swagger (/api/docs)

Para asegurar una integración óptima con las aplicaciones cliente (Web y Móvil) y posibilitar la auditoría en tiempo real, el backend incorpora la especificación formal ****OpenAPI 3.0**** servida mediante ****Swagger UI****:

- **Punto de Acceso:** Disponible en `http://localhost:3000/api/docs`.
- **Autenticación Interactiva Bearer JWT:** Esquema `JWT-auth` que permite ingresar credenciales de acceso institucional y persistirlas entre recargas de página (`persistAuthorization: true`).

- **Etiquetado Semántico (@ApiTags):** Endpoints clasificados en 13 grupos: *Sistema, Autenticación, Usuarios, Becarios, Iglesias, Lugares de Práctica, Asistencia, Pasajes, Evaluaciones, Prácticas, Notificaciones, Dashboard* y *PDF*.
- **Anotación Completa de DTOs:** Cada atributo cuenta con `@ApiProperty` o `@ApiPropertyOptional`, especificando tipos, descripciones institucionales, restricciones de rango y ejemplos funcionales.
- **Herencia Tipada en Modificaciones:** DTOs de edición parcial (`ActualizarBecarioDto`, etc.) utilizan `PartialType` de `@nestjs/swagger` para mantener la sincronización automática de metadatos.

6. Aseguramiento de Calidad y Verificación

El backend cuenta con certificación técnica continua:

- **Compilación Limpia:** `npm run compile` empaqueta la solución en 4 segundos mediante Webpack 5 y Nest CLI sin advertencias de tipado.
- **Suite de Pruebas Unitarias:** `npm test` ejecuta 7 suites con 16 pruebas aprobadas al 100 % cubriendo la aritmética de centavos, la regla del día 24, el cálculo 360° y la generación de informes.